

1. Generazione delle mesh

Gli scripts utilizzati si trovano nella cartella “TC_model_generator”, da aprire come progetto nell’IDE di PyCharm.

1.1 Generazione della configurazione base a partire dalle immagini TC

Lo script da utilizzare è “Mesh_generation.py”.

- **Righe 9 -13:** sono definite le liste relative alle singole vertebre da ricostruire, ovvero prima le lombari e poi le toraciche (trattate separatamente per poi essere unite alla fine in un’unica mesh).
Viene definito il percorso del file nifti contenente la TC della colonna vertebrale. Le immagini si trovano nella cartella “TC_images”. L’immagine considerata appartiene alla serie numero 25. Per cambiare il numero di serie sostituire nel nome del percorso tale numero con uno dei seguenti: ‘00, 14, 20, 21, 27’.
- **Righe 16-27:** Lettura dell’immagine Nifti 3D, definizione di una soglia iniziale per l’individuazione della prima vertebra (pari al massimo delle label per identificare la L1), e definizione di un voxel_size utile per eliminare componenti troppo piccole rappresentative di regioni sconnesse nella successiva ricostruzione 3D della mesh.
- **Righe 29-53:** primo ciclo per la segmentazione delle vertebre lombari. Si individua una maschera per le regioni dell’immagine la cui intensità è esattamente uguale alla soglia fissata. Segue nelle tre righe successive l’eliminazione di componenti sconnesse le cui dimensioni sono al di sotto del voxel_size.
Si ha poi la ricostruzione della mesh corrispondente alla vertebra in analisi mediante l’algoritmo del marching cubes. Tale mesh è poi opportunamente regolato, applicando uno smoothing leggero per rimuovere le componenti rumorose, e calcolando le normali ai triangoli e le loro orientazioni.
Infine, vengono definiti gli indici separatori di tale mesh, ovvero vengono identificati il punto di inizio e il punto finale di tale mesh, necessari in futuro per identificare la vertebra all’interno della nuvola punti dell’intera colonna vertebrale. Si aggiornano la mesh totale riguardo le vertebre lombari (aggiungendo progressivamente le mesh delle singole vertebre), il nuovo punto di inizio, coincidente con il punto successivo all’ultimo punto della mesh corrente, e la soglia di segmentazione (diminuisce di un’unità per identificare la nuova vertebra alla successiva iterazione).
- **Righe 55-79:** stesso identico procedimento, seguito però per le vertebre toraciche. L’unica differenza consiste nel creare un’unica mesh contenente tutte e 12 le vertebre toraciche. I separatori continuano ad essere calcolati per ogni singola vertebra.
- **Righe 83-95:** Unione delle mesh delle vertebre lombari e delle vertebre toraciche a formare quella dell’intero modello di colonna vertebrale. Visualizzazione 3D della mesh ottenuta.
- **Riga 104- :** Salvataggio delle singole mesh delle vertebre lombari, della mesh del torace e della mesh dell’intero modello. Tali mesh sono salvate nella cartella “Dataset\Meshes\BasePose25” oppure “BasePoseNN”, se si considera un numero di serie diverso da 25.
Segue il salvataggio nella stessa cartella di un file di testo in cui sono riportati per ogni vertebra l’indice di inizio e l’indice finale (separatori).

1.2 Adattamento del modello

Permette di individuare il fattore di scala che permette di sovrapporre ogni nuvola punti ottenuta dalla TC, con quella dell'intero modello OpenSim. Lo script di riferimento è "new_spine.py".

- **Righe 17 – 39:** caricamento delle nuvole punti associate al modello OpenSim (si trovano nella cartella "OpenSim_base_pose") e quelle associate al modello TC (si trovano nella cartella "Dataset\Base_meshes\Model{numero di serie}").
- **Righe 41- 51:** calcolo del fattore di scala come rapporto delle distanze medie dei punti dal centroide della nuvola punti, tra la nuvola punti TC e quella OpenSim. Segue poi la visualizzazione della nuvola punti OpenSim ingrandita di tale fattore di scala assieme a quella TC, verificandone la loro sovrapponibilità.
- **Righe 54-56:** salvataggio del fattore di scala calcolato nel file di testo info (utilizzato in precedenza per i separatori delle nuvole punti delle singole vertebre) usando la modalità "append" per evitare di eliminarne il contenuto già esistente.

2. Generazione dei dataset

Aprire un nuovo progetto selezionando la cartella "Dataset_processing".

2.1 Generazione delle nuvole punti mobili

Utilizzare lo script "generate_configuration.py".

- **Righe 6-11:** definizione dei parametri fondamentali, ovvero i numeri di serie associate a ciascun modello di colonna vertebrale ottenuto da TC con relativi fattori di scala, e labels per definire i tre diversi gradi di deformazione iniziale
- **Righe 14-77:** definizione delle informazioni utili per generare la configurazione deformata a partire da quella di riferimento: pesi associati alle rotazioni elementari di ogni vertebra per ogni grado di libertà del modello, giunti associati a ciascuna vertebra con rispettive posizioni ed orientamenti rispetto alla stessa vertebra.
- **Righe 79-108:** definizione delle funzioni per creare le matrici di trasformazioni che saranno utilizzate per generare la nuvola punti mobile.
- **Righe 111-158:** funzione generatrice della nuvola punti mobile, definita quella fissa. All'interno sono individuati degli specifici intervalli che regolano il valore dei gradi di libertà sulla base dell'entità delle tre classi deformazioni iniziali (labels), restituendo un valore casuale compreso nell'intervallo definito.
La funzione restituisce la nuvola mobile, i valori dei tre gradi di libertà utilizzati e il campo di spostamento iniziale tra la nuvola fissa e la nuvola mobile.
- **Righe 160-162:** funzione per il sottocampionamento basato su voxel delle nuvole punti.
- **Righe 165-194:** Generazione e salvataggio della configurazione base sottocampionata. Inizia il ciclo più esterno che itera sulle varie serie relative alle diverse immagini TC. Viene definita la cartella in cui trovare le mesh associate alle singole vertebre nella configurazione base, ovvero "Dataset\\Base_meshes\\Model{numero di serie}". Segue poi un primo ciclo indentato in cui all'interno di tale cartella vengono letti i file .ply associati alle mesh delle singole vertebre. Ottenute tali nuvole punti, queste sono allineate con il sistema di riferimento di Open3D, sottocampionate con un voxel_size di 5 mm, in modo da ottenere nuvole con un massimo numero di punti pari a 10000 e unite a formare la nuvola punti base dell'intera colonna vertebrale. (NB: se si vogliono ottenere le nuvole punti non sottocampionate da

utilizzare per il test sugli algoritmi tradizionali, si può eliminare il sottocampionamento ottenendo le nuvole punti originali).

- **Riga 190-:** Siamo ancora nel ciclo principale. Si crea una cartella output, posta in questo caso "Dataset\\Clouds_for_leopard\\Model_{numero di serie}". In tale cartella viene salvata la nuvola fissa unificata con il nome di "base.ply".

Seguono due cicli annidati. Il primo scorre sui diversi gradi di deformazione iniziale, e il secondo genera 200 diverse configurazioni per ognuno di tali gradi. Nella cartella output (la stessa di prima) vengono salvate le nuvole mobili con il nome "{grado di deformazione}{numero di configurazione}" e un file di testo con lo stesso nome + "_Info", in cui sono salvati i valori dei 3 gradi libertà per tale configurazione e il campo di spostamento tra la nuvola mobile ottenuta e quella fissa.

Ad esempio la 18° configurazione mobile associata a piccole deformazioni iniziali sarà salvata nel file "low17.ply" (ricordando che il contatore parte da 0) e il relativo file di testo sarà "low17_Info.txt".

Terminata l'esecuzione dello script avremo all'interno della cartella "Clouds_for_leopard" delle sottocartelle relative a ciascuna serie TC in cui sono contenute le nuvole punti fisse, le nuvole mobili e i relativi file di testo contenenti utili informazioni.

2.2 Generazione del dataset per LePard

Lo script di riferimento è "generate_dataset.py".

- **Righe 7-12:** definizione dei parametri fondamentali che permettono di identificare le nuvole punti, ovvero i modelli con relative scale e sottogruppi di destinazione (del dataset), e classi di deformazione. Viene anche definita la cartella di destinazione, ovvero "C:\\Users\\olocc\\OneDrive\\Desktop\\Thesis_new\\Dataset\\4D_Match_dataset_heavy\\split\\split".
- **Righe 14-20:** primo ciclo esterno che costruisce la cartella di ricerca in cui sono presenti le diverse nuvole punti per ciascun modello di colonna vertebrale. È ottenuta come "C:\\Users\\olocc\\OneDrive\\Desktop\\Thesis_new\\Dataset\\Clouds_for_leopard\\Model" + {serie}, dove {serie} rappresenta il numero relativo al modello di colonna vertebrale. Viene poi letta la configurazione mobile e creata la cartella output in cui saranno salvati i diversi file per il modello considerato.
- **Righe 21-:** doppio ciclo che scorre sulle diverse classi di deformazione e sulle diverse configurazioni per classe. Al suo interno viene dapprima letta la nuvola mobile corrispondente alla label e alla configurazione considerata. Si convertono in array i punti delle nuvole fissa e mobile e li si trasla entrambi rispetto al centroide della nuvola mobile. Dal file "Info" associato alla nuvola mobile viene letto il campo di spostamento che lega la nuvola fissa alla mobile e lo si salva in una matrice. È stato verificato che applicando tale campo alla nuvola fissa, si ottiene una nuvola coincidente con quella mobile. Segue poi il calcolo della matrice di rotazione e del vettore traslazione, posti rispettivamente pari all'identità e al vettore nullo, non essendoci roto-traslazioni globali. Infine, segue il calcolo delle corrispondenze, del tasso di sovrapposizione e dell'indice di metrica in accordo con le funzioni precedentemente elencate. Le **righe 55-66** prevedono il salvataggio delle informazioni calcolate in un file .npz, prestando attenzione ad usare dati di tipo coerente con quelli del Dataset originale di Leopard.

Al termine dell'esecuzione del codice si ottiene una cartella "4D_Match_dataset_heavy\\split\\split" contenente le tre cartelle di train, validation e test ognuna con delle sottocartelle relative ai propri

modelli di colonna vertebrale, ciascuna delle quali contiene i file .npz relativi a ciascuna coppia di nuvole fissa e mobile.

2.3 Calcolo dei volume bounds

Lo script di riferimento è “Compute_yaml_parameters”.

I volume bounds necessari per addestrare il modello Leopard sono calcolati mediante la funzione presente alle righe 11-45. In particolare i valori minimi lungo le tre direzioni vengono inizializzati ad infinito, mentre quelli massimi a meno infinito. Successivamente sono presenti tre cicli annidati, di cui il primo itera sui diversi modelli di colonna vertebrale, il secondo su ciascuna classe di deformazione e il terzo su ciascuna configurazione. Per ognuna di esse si ottengono i punti della nuvola sorgente e della nuvola target, calcolando i valori massimi e minimi lungo ciascuna delle tre direzioni considerando entrambe le nuvole punti. Se il valore massimo ottenuto per una determinata direzione supera quello attuale, allora il massimo assoluto associato a tale direzione viene sostituito con il valore calcolato. In modo del tutto analogo si ottiene il minimo assoluto per ciascuna direzione.

Nelle righe 56-58, viene chiamata tale funzione e vengono stampati i bounds ottenuti.

3. Algoritmi di registrazione

Utilizzare come progetto la cartella “TC_model_registration”.

3.1 Registrazione rigida (RANSAC + ICP)

Lo script di riferimento è contenuto nel file “Rigid_ICP.py”.

I file che saranno utilizzati per testare l'algoritmo sono contenuti nella cartella “Dataset//For_tests//Model27”, in cui sono presenti la nuvola punti base relativa alla 27° serie TC e 25 diverse configurazioni per grado di deformazione iniziale. Tali nuvole non sono sottocampionate, per cui contengono i punti originali.

- **Righe 9-13:** definizione dei parametri principali: fattore di scala, labels e numero di configurazioni per grado di deformazione iniziale (25).
- **Righe 16-30:** Funzione per la visualizzazione delle nuvole punti in momenti diversi della registrazione (non utilizzata per condurre i test).
- **Righe 33-49:** funzioni per il sottocampionamento delle nuvole punti fissa e mobile, e per l'estrazione delle feature FPFH, utili per eseguire l'algoritmo RANSAC.
- **Righe 52-65:** funzione per l'esecuzione di RANSAC. Restituisce la matrice di trasformazione omogenea ottenuta mediante tale algoritmo.
- **Righe 68-73:** funzione per l'esecuzione dell'algoritmo ICP. Restituisce la matrice di trasformazione omogenea ottenuta mediante tale algoritmo.
- **Righe 76-121:** funzioni per il calcolo delle metriche di valutazione: errore globale di registrazione (global_metrics), errore di deformazione (distance_preservation_error) e metriche punto-a-punto (distance_metrics).
- **Righe 131-153:** definizione dei percorsi in cui si trovano la nuvola punti fissa (“Dataset\For_tests\Model27\base.ply”) e il file di testo in cui sono contenuti i separatori relativi a ciascuna vertebra del modello (ottenuto nel paragrafo 1.1 -> “Dataset\Base_meshes\Model27\Infos.tx”).

Definizione dei percorsi dei file di testo in cui saranno salvati i risultati prima e dopo la registrazione, nella cartella “Results” sita nella stessa cartella del progetto. Scrittura degli

header in tali file testuali, in modo da individuare in maniera più semplice le metriche utilizzate e i relativi valori a fine registrazione.

Salvataggio degli indici separatori (per l'individuazione delle singole vertebre nell'intera nuvola punti) in una matrice 17×2 , dove le righe corrispondono ad ogni vertebra ordinata e le colonne individuano rispettivamente il punto di inizio e il punto di fine per ognuna di esse. Tali indici sono importati dal file di testo generato in precedenza al paragrafo 1.1.

- **Riga 156 - :** Ciclo principale. Esso è composto da un ciclo esterno che itera per le 3 classi di deformazione iniziale, ed uno interno che itera sul numero di configurazioni per ogni classe (25). Ad ogni iterazione viene letta la nuvola mobile corrispondente alla classe e al numero di configurazione (quelle generate nel paragrafo 2.1).

Con tali nuvole si calcolano le metriche di valutazione iniziali e le si salvano nel file di testo "Pre.txt" presente nella cartella "Results". Tali metriche sono salvate nell'ordine previsto dal relativo header.

Segue un loop che parte dalle nuvole punti unificate (fissa e mobile relative all'intera colonna vertebrale), le suddivide nelle singole vertebre lombari e nel torso (insieme delle 12 vertebre toraciche), e le salva ognuna all'interno di un buffer, creato sia per la nuvola fissa che per quella mobile. Tale passaggio è fondamentale per eseguire la successiva registrazione rigida vertebra per vertebra.

Segue un terzo ciclo annidato che scorre su ogni coppia di vertebra fissa e mobile eseguendo la registrazione rigida dapprima sottocampionando tali nuvole, poi applicando l'algoritmo RANSAC e infine applicando il raffinamento con ICP. Viene calcolata la matrice di trasformazione omogenea ottenuta da tale pipeline e applicata sulla nuvola mobile. Le nuvole trasformate con tali matrici sono salvate nel buffer "src_pc_buffer_icp", che viene assunto come riferimento per la valutazione finale dei risultati ottenuti.

Terminato tale ciclo viene calcolata e stampata la metrica di valutazione globale prima e dopo la registrazione.

Prima di procedere con la valutazione dei risultati è presente un ciclo che permette di unificare le nuvole trasformate in un'unica nuvola punti, in quanto le metriche di valutazione lavorano solo su un'unica nuvola punti.

Infine, si calcolano le diverse metriche di valutazione, salvandole nel file di testo "RANSAC+ICP.txt" presente nella cartella "Results". Tali metriche sono inserite nello stesso ordine previsto dall'header (definito inizialmente).

3.2 Registrazione non rigida con CPD

Lo script a cui si fa riferimento è "NonRigid_CPD.py".

- **Righe 11-190:** funzioni analoghe a quelle viste in precedenza per RANSAC+ICP. Le differenze principali sono:
 - **Righe 45-53:** funzione per la normalizzazione delle nuvole punti (utile per agevolare il processo di registrazione non rigida). Essa calcola il centroide della nuvola punti e un fattore di scala, inteso come la massima dimensione della nuvola punti. Tale nuvola è poi centrata nell'origine del sistema di riferimento globale e riscalata per tale fattore. In uscita vengono restituiti la nuvola punti normalizzata, il centroide (offset) e il fattore di scala.
 - **Righe 55, 56:** Funzione per la denormalizzazione delle nuvole punti. Serve per riportare le nuvole punti alle dimensioni originali, utilizzando il centroide e il fattore di scala ottenuti nella fase di normalizzazione.
 - **Righe 84 -150:** funzione per l'esecuzione ottimizzata della registrazione. Essa esegue più cicli di registrazione tentando di adattare dinamicamente i parametri tipici

dell'algoritmo alle nuvole punti che devono essere allineate, restituendo il risultato ottimale di registrazione. In ingresso necessita dell'errore iniziale (necessario per il calcolo dei parametri), delle due nuvole punti da allineare e dei parametri di denormalizzazione (centroide e scala).

In uscita restituisce la nuvola punti deformata, risultato del processo di ottimizzazione.

- **Righe 152-161:** funzione per la propagazione della deformazione. Permette di estendere la deformazione a tutti i punti della nuvola originale partendo da quella sottocampionata, utilizzando un'interpolazione lineare basata sulla pesatura della distanza inversa, in modo che punti più vicini abbiano spostamento simile tra loro.
- **Righe 194-213:** Stessa procedura vista nel caso RANSAC+ICP. Definizione dei percorsi da cui leggere la nuvola punti base e ottenere gli indici separatori (gli stessi utilizzati per RANSAC+ICP), definizione del percorso in cui salvare il file di testo contenente i risultati di fine registrazione ("CPD.txt" nella cartella "Results"), scrittura dell'header in tale file e salvataggio in un array degli indici separatori.
- **Righe 217-:** ciclo di registrazione. Si compone di un ciclo esterno che itera sulle diverse classi di deformazione e uno interno che itera sulle diverse configurazioni per ogni classe. Definizione del percorso in cui trovare la nuvola punti mobile, lettura delle nuvole punti fissa e mobile, valutazione e stampa dell'errore globale prima della registrazione. Da questo punto in poi si fissa l'istante in cui inizia il processo di registrazione. Segue un sottocampionamento basato su voxel di entrambe le nuvole punti (analogo a quello utilizzato per la registrazione rigida) e un pre-allineamento rigido con RANSAC della nuvola punti fissa, se il suo scostamento iniziale dalla nuvola mobile supera 60 mm. Successivamente entrambe le nuvole punti sottocampionate sono normalizzate, calcolando anche l'offset e la scala di normalizzazione. Terminate tali operazioni inizia il ciclo ottimizzato di registrazione, passando alla relativa funzione (righe 84-150) le variabili relative all'errore globale iniziale, le nuvole fissa e mobile normalizzate e il centroide e la scala della nuvola mobile come parametri di denormalizzazione. Al termine di tale operazione si ottiene la nuvola fissa deformata adattata alla nuvola mobile sottocampionata. Si crea dapprima un oggetto `point_cloud` vuoto che sarà popolato da un insieme di punti in numero esattamente uguale a quello della nuvola fissa originale. Segue poi come ultimo step la propagazione della deformazione, passando alla relativa funzione (righe 152-161) le variabili relative ai punti delle nuvole fissa sottocampionata, fissa deformata sottocampionata e fissa deformata. Terminata tale operazione viene salvato l'istante corrente come istante di fine registrazione e viene calcolato il tempo totale di registrazione come l'intervallo intercorso tra l'istante di inizio registrazione e quello di fine registrazione. Infine, si calcolano le metriche di registrazione e le si salvano in un file di testo nell'ordine previsto dall'header scritto inizialmente. Al termine dell'esecuzione dello script nella cartella "Results" sarà presente un file testuale denominato "CPD.txt" che conterrà tali risultati.

3.3 Registrazione con GMMReg

Lo script di riferimento è "GMMReg.py".

- **Righe 1-144:** stesse funzioni utilizzate nel caso della registrazione non rigida con CPD. L'unica differenza è nella funzione compresa tra le **righe 85-102**, in cui sono calcolati i 4 parametri caratteristici dell'algoritmo secondo leggi euristiche. L'obiettivo è stato evitare cicli di ottimizzazione che avrebbero comportato grandi incrementi dei tempi di esecuzione dell'algoritmo.
- **Righe 148-168:** Stessa procedura del workflow di registrazione con CPD: prima sono definiti i percorsi in cui si trovano la nuvola punti fissa e il relativo file di testo con gli indici separatori,

poi viene letta la nuvola fissa, poi si definisce un percorso al file di testo in cui salvare i risultati di registrazione (“GMMReg.txt” presente ancora nella cartella “Results”), e al suo interno si scrive un header contenente la nomenclatura delle metriche che saranno valutate, ed infine si leggono gli indici separatori dal relativo file di testo salvandoli in un array.

- **Righe 170-:** Ciclo di registrazione, del tutto analogo a quelli sinora visti. Sono presenti sempre due cicli, quello esterno che itera su ogni classe di deformazione iniziale e quello interno che itera sulle diverse configurazioni per classe. Le operazioni compiute sono identiche a quelle compiute nella registrazione con CPD. Le uniche differenze sono alla **riga 198**, dove sono calcolati dinamicamente i parametri di registrazione e la chiamata, alle **righe 200-203**, della funzione “registration_gmmreg” del pacchetto “l2_dist_regs” della libreria “probreg”, in cui sono passati i punti della nuvola fissa normalizzata e della nuvola mobile normalizzata, e i parametri precedentemente calcolati.

Al termine dell’esecuzione comparirà un file “GMMReg.txt” nella cartella “Results” in cui possono essere visualizzati i risultati di registrazione.

3.4 Registrazione con SVR

Lo script di riferimento è “SVR.py”.

- **Righe 1-148:** stesse funzioni utilizzate per le registrazioni con CPD e GMMReg. L’unica differenza è visibile alle righe **98-107**, in cui è presente una funzione per il calcolo dinamico dei due parametri principali di registrazione al variare dell’errore globale iniziale.
- **Righe 152-172:** stessa procedura seguita nei due casi di registrazione non rigida esposti in precedenza, ovvero lettura della nuvola punti fissa, definizione del file di testo in cui saranno salvati i risultati a fine registrazione (“SVR.txt” nella cartella results) e scrittura al suo interno dell’header che definisce le metriche che saranno calcolate. Lettura degli indici separatori dal relativo file di testo e salvataggio in un array di dimensione 17×2 .
- **Riga 175-:** ciclo di registrazione, identico a quello utilizzato per le registrazioni CPD e GMMReg. L’unica differenza è presente nelle righe 204-213, in cui dapprima vengono calcolati i parametri di registrazione (funzione definita alle righe 98-107) in funzione dell’errore iniziale di registrazione, e la chiamata alla funzione “registration_svr” del modulo “l2_dist_regs” della libreria Probreg.

4. Processing dei dati ottenuti da Leopard

La cartella contenente gli script per processare i dati provenienti da Leopard è “Leopard_post_processing”.

4.1 Generazione di nuvole trasformate a media risoluzione

Lo script di riferimento è “med_res_post_clouds_generator”, e permette di ottenere nuvole deformate dalle trasformazioni calcolate con Leopard aventi la stessa risoluzione di quelle presenti nel dataset.

- **Righe 6-15:** funzione per la propagazione della deformazione a nuvole punti con risoluzione più alta. È la stessa funzione utilizzata per gli algoritmi tradizionali di registrazione non rigida.
- **Righe 17-19:** definizione delle cartelle necessarie per il processing, ovvero la cartella contenente le nuvole sorgenti a media risoluzione, la cartella contenente i risultati di registrazione ottenuti dai test di Leopard, e la cartella in cui saranno salvate le nuvole punti deformate a media risoluzione.
- **Righe 21-25:** lettura della nuvola punti sorgente a media risoluzione.

- **Righe 27-:** ciclo principale. Per ogni iterazione viene letto il file relativo ai risultati di predizione restituiti da Lepard all'interno della cartella definita precedentemente. Tale file ha nome del tipo "prediction_{numero predizione}.npz". Per ciascuno di tali file vengono letti i dati contenuti al suo interno. Dapprima si ricavano i punti della nuvola source, costruendone la nuvola punti associata. Essa viene poi traslata di un offset pari alla distanza tra i centri della nuvola source originale e la nuvola source ottenuta da Lepard. Alle righe 41-51 vengono ricavate le trasformazioni stimate dal modello per le nuvole a bassa risoluzione e vengono applicate alla nuvola source a bassa risoluzione. Infine si ha la propagazione di tali trasformazioni anche alla nuvola source a media risoluzione, costruendone una nuvola punti ad essa associata. Quest'ultima viene salvata nella cartella di uscita con un filename del tipo "prediction_{numero predizione}.ply".

4.2 Conversione delle corrispondenze predizione-modello

Lo script di riferimento è "NPZ_to_MAT", e permette di convertire il contenuto di un file NPZ in un file MAT più facilmente trattabile dall'ambiente Matlab.

Il file che viene letto è "corresponances.npz" e contiene al suo interno una lista che associa a ciascuna predizione svolta dal tester di Lepard la relativa configurazione del modello utilizzato per il test. Gli elementi che si ottengono sono del tipo [{"numero predizione"}, {"grado di deformazione"}{numero configurazione}]. Lo script inverte l'ordine degli elementi nella lista e li salva in un file "correspondences.mat".

4.3 Allineamento non rigido con N-ICP

Lo script di riferimento è "N_ICP_refinement". È un file con estensione ".m", e pertanto deve essere eseguito in ambiente Matlab.

- **Prima sezione (righe 1-6):** pulizia del workspace e della command window, e successivo caricamento del cell array relativo alle corrispondenze predizione-configurazione (precedentemente convertito da NPZ a MAT).
- **Seconda sezione (righe 7-57):** definizione della cartella "Dataset\Lepard_nr_icp_results" in cui saranno salvati i risultati di registrazione e inizializzazione del cell array times che conterrà per ciascun file di predizione il tempo di esecuzione impiegato dall'algoritmo. Segue poi un ciclo che itera su ciascuna coppia predizione-configurazione, considerando le nuvole source (nuvole a media risoluzione ottenute al paragrafo 4.1) e target (nuvole target utilizzate per il test di Lepard). Ognuna di esse viene trasformata in una struct contenente i campi faces (lasciato vuoto in quanto le nuvole punti, diversamente dalle mesh, non presentano connettività e superfici), vertices (coordinate dei punti delle nuvole), e normals (normali ai punti di ciascuna nuvola).
Segue il settaggio delle opzioni di registrazione, come la possibilità di utilizzare le normali ai punti, di plottare in tempo reale le deformazioni subite dalla nuvola source durante la registrazione, e di utilizzare dei landmarks se disponibili.
Alla riga 47 viene eseguita la procedura vera e propria di registrazione, restituendo in uscita i punti trasformati della nuvola source e la matrice di trasformazione source-target.
Infine le coordinate dei punti trasformati sono salvati nella cartella finale come file avente nome "{grado di deformazione}{numero configurazione}.mat". Alle righe 55-56, invece viene popolato il cell array times.
- **Righe 59-60:** salvataggio dei tempi di esecuzione in un file "times.mat" presente nella cartella di destinazione definita prima del ciclo principale.

4.4 Estensione delle deformazioni alle nuvole ad alta risoluzione

Lo script di riferimento è “High_resolution_post_NICP”, e permette di estendere le trasformazioni ottenute nel paragrafo 4.3, alle nuvole originali ad alta risoluzione.

- **Righe 7-19:** definizione dei parametri fondamentali (grado di deformazione e numero di configurazioni), e funzione per la propagazione delle deformazioni a nuvole punti con maggior risoluzione.
- **Righe 22-25:** lettura della nuvola source ad alta risoluzione.
- **Righe 27-30:** lettura della nuvola source a media risoluzione (utilizzata per il test di Leopard).
- **Riga 33:** definizione della cartella in cui salvare le nuvole punti trasformate ad alta risoluzione.
- **Righe 36-:** ciclo di iterazione per ciascun grado di deformazione e per ciascuna configurazione. Al suo interno si ha la lettura del file MAT contenente i punti trasformati associati a tale configurazione, e la successiva creazione della nuvola contenente tali punti (a media risoluzione).

Alle righe 49-58 viene chiamata la funzione che permette di estendere la trasformazione anche alla nuvola source ad alta risoluzione.

Infine la nuvola ottenuta viene salvata nella cartella di destinazione con un filename del tipo “{grado di deformazione}{numero configurazione}.ply”.

4.5 Calcolo del tempo totale di esecuzione

Lo script di riferimento è “Total_time.py”.

- **Righe 4-13:** Definizione del filename relativo al file MAT contenente i tempi di esecuzione di N-ICP per gli esempi di Leopard (“Leopard_nr_icp_results\times_nicp.mat”). Caricamento del file MAT, lettura della prima colonna presente al suo interno (contenete i valori associati ai diversi tempi di esecuzione) e creazione di un dizionario che associa a ciascuna predizione di Leopard, i tempi di esecuzione richiesti da N-ICP.
- **Righe 15-24:** Calcolo del tempo totale di esecuzione per ciascun esempio. In particolare è presente un ciclo che itera su ciascuna predizione di Leopard, leggendo i diversi file contenenti i tempi di esecuzione richiesti da esso. Tali tempi sono sommati poi con quelli richiesti da N-ICP per i campioni corrispondenti, ed infine viene creato un dizionario che associa a ciascuna predizione il relativo tempo totale di esecuzione.
- **Righe 26-34:** Caricamento del file NPZ contenente le corrispondenze tra il numero di predizione e la relativa configurazione deformata, e creazione di un dizionario associato. Segue un ciclo che itera su ciascuna predizione. Al suo interno sono ricavati la configurazione deformata associata alla predizione corrente, il tempo totale di esecuzione associato alla configurazione corrente, e viene creato un dizionario che mette in relazione la configurazione deformata con il relativo tempo totale di esecuzione. ATTENZIONE, il dizionario non è ordinato, ovvero l'ordine con cui sono salvati i valori non rispetta quello delle labels [‘low’, ‘medium’, ‘high’] + [0,...,29].
- **Righe 36-:** salvataggio dei tempi totali di esecuzione “lep_n_icp_times”, mediante due cicli annidati che seguono l'ordine definito al punto precedente. Il vettore è salvato in un file “post_nicp_times.npz” nella cartella “Leopard Tests”.

4.6 Calcolo delle metriche di registrazione

Lo script di riferimento è “Leopard_results_generator”.

- **Righe 7-101:** definizione dei parametri fondamentali, e delle funzioni necessarie per il calcolo delle metriche di valutazione.
- **Righe 104-110:** definizione delle cartelle contenenti la nuvole source (ottenute al paragrafo precedente) e target ad alta risoluzione. Successivamente viene letta la nuvola source originale (non deformata).
- **Righe 113-123:** Lettura degli indici separatori delle singole vertebre.
- **Righe 126-129:** definizione del file contenente i risultati di registrazione ("Lepard.txt" nella cartella Results), e scrittura al suo interno di un header in cui vengono definite le metriche utilizzate.
- **Righe 131-:** doppio ciclo in cui vengono lette le nuvole source trasformate (ottenute al paragrafo precedente) e le nuvole target. Segue, note tali nuvole, il calcolo delle metriche di registrazione mediante le apposite funzioni, e la loro scrittura, nel file di testo definito in precedenza, secondo l'ordine previsto dall'header.

Box plot dei risultati finali

Lo script di riferimento è "Box_lpots.py" presente nella cartella "TC_model_registration".

- **Righe 7-18:** definizione dei parametri fondamentali (gradi di deformazione e numero di configurazioni per grado di deformazione). Definizione della cartella base contenente i file con i risultati di registrazione. Creazione di un dizionario che associa a ciascun metodo il relativo file di testo contenente i risultati di registrazione (presente nella cartella base).
- **Righe 20-25:** estrazione delle metriche di valutazione presenti per ciascun metodo ("method_metrics"), lettura dei dati dai relativi file di testo, e trasformazione di questi ultimi in dizionari contenenti matrici (data).
- **Righe 27-48:** Definizione di parametri importati per i successivi plot: titolo per l'asse y ("plot_labels"), scala da utilizzare nel plot ("plot_scales") e titolo della figura associato alla metrica considerata ("metric_labels").
- **Righe 50-101:** definizione dei parametri importanti per i boxplot: lista delle metriche, lista dei metodi e lista dei colori associati ai box di ciascun metodo.
Segue il ciclo principale che genera un boxplot per ogni metrica. Al suo interno per ogni grado di deformazione e per ogni metodo, viene calcolato lo slice per ogni livello, assumendo che ciascuno di essi contenga 30 diversi esempi per metrica. Viene salvato inoltre l'indice "idx" della metrica corrente presente nel file del metodo considerato. Successivamente sono disegnate due linee verticali per separare i tre livelli di deformazione.
Alla riga 72 vengono realizzati i boxplot per la metrica considerata.
Alle righe 73-84, sono presenti le impostazioni di posizione testo per inserire i titoli che permettono di identificare le classi di deformazione all'interno della figura.
Alle righe 85-87 vengono poi colorati i vari box associati ai diversi metodi e viene colorata la linea mediana in ognuno di essi.
Alle righe 88-97, sono presenti impostazioni relative ai titoli lungo gli assi x (carattere e rotazione) e y, su cui viene anche adottata la relativa scala del plot.
Alle righe 99-101, infine la figura viene salvata in un file denominato "BP-{metrica}.jpg", all'interno della stessa cartella in cui sono presenti i file di testo relativi ai risultati.
- **Righe 104-:** realizzazione dei boxplot relativi alla metrica del tempo di esecuzione seguendo le stesse modalità finora esposte (naturalmente non viene eseguito nessun ciclo sulle metriche, dato che la metrica considerata è solo una). La differenza sostanziale è dovuta al fatto che il tempo di esecuzione per il metodo "Lepard+N-ICP" non è presente nel relativo file di testo, ma

viene letto dal file NPZ ottenuto nel paragrafo 4.5. Alle righe 106-110 viene letto il contenuto di tale file ed inoltre viene aggiunta la metrica “Time” alla lista delle metriche disponibili per il metodo Lepard+N-ICP. Viene, invece, settata a “None” la metrica del tempo per il metodo “Pre”.